

Safe Edges

PRELIMINARY AUDIT REPORT

2025



SAFEEDGES.IN

About SafeEdges

SafeEdges is a vulnerability research firm with deep expertise in blockchain security. We specialize in **EVM**, **Move (Aptos and Sui)**, **Solana**, as well as **Cairo**, **NEAR**, and **Cosmos**.

We review L1s and L2s, cross-chain protocols, wallets, applied cryptography, zero-knowledge circuits, web applications, and more. Our engineers bring a rich set of skills, including cryptography, web security, mobile security, low-level exploitation, and finance.

Our background in traditional information security and public bug bounty programs enables us to consistently discover hidden vulnerabilities and develop novel security research, earning a reputation as a trusted security partner for teams whose rate of innovation outpaces the existing security landscape.

- safeedges.in - [@safe-edges](https://twitter.com/safe-edges) - info@safeedges.in
-

Table of Contents

1. [Executive Summary](#)
2. [Scope](#)
3. [Methodology](#)
4. [Severity Classification](#)
5. [Findings Overview](#)
6. [Detailed Findings](#)
7. [Conclusion](#)
8. [Disclaimer](#)

Protocol Summary

Podha Protocol is a multi-strategy asset management and yield generation platform. It enables users to allocate capital in a permissionless manner to a diversified yield engine, earning sustainable returns and Drop Points over time.

The platform caters to family offices, institutions, high-net-worth individuals (HNWIs), and retail investors. Podha aims to offer competitive yields through a combination of strategies, including market-neutral, arbitrage, directional, and real-world asset (RWA) integrations.

Podha's vision is to continuously expand vault offerings and allow every user to manage their portfolio within a single, unified platform without overpromising or guaranteeing returns.

The platform runs a multi-strategy yield engine that integrates both centralized and decentralized opportunities, including:

- **Arbitrage:** Intra-exchange arbitrage between spot and futures markets.
- **Funding Rate Capture:** Earning passive income from perpetual markets through delta-neutral setups.
- **Delta-Neutral Positioning:** Balanced trades that neutralize price risk while generating consistent returns.
- **Directional Strategies:** Deployed selectively based on market conviction and quantitative signals.

Content and information sourced from the official Podha Protocol documentation: [Podha GitBook](#)

Executive Summary

Item	Details
Scope	https://github.com/parth26nath/Podha_Backend
Timeframe	01/09/2025 - 11/09/2025
Methodology	Manual code review, static analysis, best practices check
Severity Summary	Low: 4
Overall Risk	⚠️ Low
Status	Acknowledged

This audit reviewed the core Podha contracts to identify potential security and logic gaps. Several low-severity issues were identified, mainly related to management fee handling, early withdrawal logic, yield tier implementation, admin functions, and frontend security. Most of these issues can be resolved with straightforward code improvements, as detailed below.

Number of Security Issues per Severity

Severity	Count	Percentage
● Critical	0	0%
■ High	0	0%
● Medium	0	0%
● Low	4	100%
● Informational	0	0%

Techniques and Methods

The audit employed a combination of automated and manual techniques to ensure a thorough evaluation of the Podha Protocol.

- **Code Quality Checks** – Ensuring proper code structure, readability, and maintainability.
- **Best Practices & Documentation Review** – Verifying that implementation aligns with protocol documentation and industry standards.
- **Reentrancy & ERC Compliance Checks** – Checking for reentrancy, standard token compliance (ERC20/ERC721), and security vulnerabilities.
- **Frontend & Backend Integration Review** – Analyzing interactions between the smart contracts, backend, and UI to detect logical or operational gaps.
- **Cross-Chain & Interoperability Checks** – Evaluating interactions with external contracts, bridges, and L1/L2 integrations.

Analysis Types

- **Structural Analysis** – Reviewed overall design patterns, modularity, and contract relationships.
- **Static Analysis** – Automated scanning for vulnerabilities using specialized tools.
- **Manual Analysis** – Line-by-line code review to identify logical flaws and edge-case issues.
- **Gas Consumption Analysis** – Optimizing loops, storage, and transactions for cost-efficiency.
- **Scenario & Functional Testing** – Testing expected and unexpected user actions to validate system behavior.
- **Dependency & Library Review** – Checking external libraries and dependencies for vulnerabilities.
- **Security Pattern Verification** – Confirming adherence to secure coding patterns and SWC Registry recommendations.

Tools Used:

Remix IDE, Foundry, Slither, Mythril, Solhint, Hardhat, Echidna, Oyente, Tenderly, OpenZeppelin Contracts, Truffle, Ganache, VSCode Solidity Analyzer

Note: This multi-layered methodology ensures that both low-level code issues and high-level logical flaws are identified, providing a comprehensive security assessment of the protocol.

Checked Vulnerabilities

✓	Access Management	✓	Compiler version not fixed
✓	Arbitrary write to storage	✓	Address hardcoded
✓	Centralization of control	✓	Divide before multiply
✓	Ether theft	✓	Integer overflow/underflow
✓	Improper or missing events	✓	ERC compliance
✓	Logical issues and flaws	✓	Dangerous strict equalities
✓	Arithmetic Computations Correctness	✓	Tautology or contradiction
✓	Race conditions/front running	✓	Return values of low-level calls
✓	SWC Registry	✓	Missing Zero Address Validation
✓	Re-entrancy	✓	Private modifier
✓	Timestamp Dependence	✓	Revert/require functions
✓	Gas Limit and Loops	✓	Multiple Sends
✓	Exception Disorder	✓	Using suicide
✓	Gasless Send	✓	Using delegatecall
✓	Use of tx.origin	✓	Upgradeable safety
✓	Malicious libraries	✓	Using throw

Severity Levels

Critical

Issues that can completely compromise protocol integrity or allow direct theft/loss of funds. Must be fixed before deployment.

High

Exploitable vulnerabilities affecting critical functionality, potentially causing loss of assets, broken pools, or bypassed core logic. Must be addressed immediately.

Medium

Weaknesses causing errors, inconsistent behavior, or revenue loss under certain conditions. Should be fixed but not catastrophic.

Low

Minor issues with limited impact, such as edge-case bugs or inefficient gas usage. Worth fixing for reliability.

Informational

Cosmetic or documentation issues. Do not pose direct threats but may confuse users or developers.

Issue Status

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Resolved

Security vulnerabilities that have been identified and successfully fixed or mitigated.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Findings Overview

ID	Title	Severity	Status
L1	Management fee logic could be automated	LOW	Acknowledged
L2	Early exit fees logic to match documentation	LOW	Acknowledged
L3	Multiple yield tiers can provide more flexibility	LOW	Acknowledged
L4	Admin functions currently centralized	LOW	Acknowledged

Detailed Findings

L1. Management Fee Logic Could Be Automated

Severity: Medium

Status: Noted

Observation:

Currently, management fees for users withdrawing before the 30-day lock period are applied manually. Automating this process could improve consistency and reduce manual effort.

Recommendations:

- Automate fee deduction within smart contract or backend service.
- Ensure fees are calculated and applied consistently.
- Consider flexible parameters for adjusting fees in the future.

Client Response: Automation is planned in future updates.

Example Implementation (JavaScript):

```
const MANAGEMENT_FEE_RATE = 0.01; // 1%

function calculateManagementFee(withdrawAmount) {
  return withdrawAmount * MANAGEMENT_FEE_RATE;
}

function processWithdrawal(user, withdrawAmount) {
  const fee = calculateManagementFee(withdrawAmount);
  const netAmount = withdrawAmount - fee;
  console.log(`Processed withdrawal for ${user.name}: Gross=${withdrawAmount}, F
```

L2. Early Exit Fees Logic to Match Documentation

Severity: Medium

Status: Noted

Observation:

Users can withdraw principal at any time, but documentation specifies a 2.5% early exit fee. Implementing this logic aligns platform with documentation.

Recommendations:

- Add smart contract or backend logic to apply early exit fees.
- Display early exit options and corresponding fees clearly on frontend.

Client Response: Early exit fee logic can be added in upcoming updates.

Example Implementation (JavaScript):

```
const EARLY_EXIT_FEE_RATE = 0.025; // 2.5%

function isEarlyWithdrawal(depositDate, lockPeriodDays) {
  const now = new Date();
  const lockEndDate = new Date(depositDate);
  lockEndDate.setDate(lockEndDate.getDate() + lockPeriodDays);
  return now < lockEndDate;
}

function processEarlyExit(user, withdrawAmount, depositDate, lockPeriodDays) {
  let fee = 0;
  if (isEarlyWithdrawal(depositDate, lockPeriodDays)) {
    fee = withdrawAmount * EARLY_EXIT_FEE_RATE;
  }
  const netAmount = withdrawAmount - fee;
  console.log(`Early exit withdrawal for ${user.name}: Fee=${fee}, Net=${netAmount}`);
}
```

L3. Multiple Yield Tiers Can Provide More Flexibility

Severity: Low

Status: Noted

Observation: Only a single deposit tier (\$50 minimum) is supported. Multiple tiers could offer more flexibility.

Recommendations:

- Implement multiple yield tiers (50, 150, 500, 1500).
- Ensure smart contract logic supports tiered rewards.
- Show tiers clearly on frontend.

Client Response: Multiple categories will be introduced in future updates.

Example Implementation (JavaScript):

```
const yieldTiers = [
  { minDeposit: 50, yieldRate: 0.03 },
  { minDeposit: 150, yieldRate: 0.04 },
  { minDeposit: 500, yieldRate: 0.05 },
  { minDeposit: 1500, yieldRate: 0.06 },
];

function getYieldRate(depositAmount) {
  let applicableTier = yieldTiers.slice().reverse().find(tier => depositAmount > tier.minDeposit);
  return applicableTier ? applicableTier.yieldRate : 0;
}

function calculateYield(depositAmount) {
  const rate = getYieldRate(depositAmount);
  return depositAmount * rate;
}
```

L4. Admin Functions and Centralization

Severity: Low

Status: Noted

Observation: Some critical functions are handled by a single admin, creating centralization risk.

Recommendations:

- Automate routine admin tasks.
- Implement **multi-sig approvals** for sensitive actions.
- Assign multiple roles for actions (Fee Manager, Withdrawal Approver, Auditor).
- Maintain separate audit trails.

Client Response: Automation and decentralization are planned for future updates.

Example Implementation (JavaScript):

```

const roles = {
  feeManager: [],
  withdrawalApprover: [],
  auditor: []
};

const requiredApprovals = 2;
let approvals = {};

function assignRole(role, adminId) {
  if (!roles[role].includes(adminId)) {
    roles[role].push(adminId);
  }
}

function approveAction(role, adminId, actionId) {
  if (!approvals[actionId]) approvals[actionId] = {};
  if (!approvals[actionId][role]) approvals[actionId][role] = new Set();

  approvals[actionId][role].add(adminId);

  if (approvals[actionId][role].size >= requiredApprovals) {
    executeAction(actionId, role);
  }
}

function executeAction(actionId, role) {
  approvals[actionId][role] = new Set(); // reset
}

```

Conclusion

Podha Protocol is functional with a solid foundation. Implementing the suggested improvements will:

- Automate management and withdrawal processes
 - Introduce early exit fees to match documentation
 - Expand yield tiers for flexibility
 - Streamline admin tasks with multi-sig, multi-role, and audit trails
-

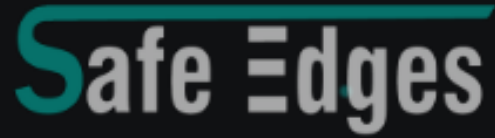
Disclaimer

This assessment does not provide any warranties regarding the identification of all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. SafeEdges also cannot provide guarantees about any code added to the project after the version reviewed during our assessment.

Because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, SafeEdges provides a recommended solution. All code samples in these recommendations are intended to illustrate how an issue may be resolved (i.e., the concept) and may not be tested or fully functional. These recommendations are not exhaustive, and we encourage partners to consider them as a starting point for further discussion. SafeEdges is happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only. Do not construe any information in this report as legal, tax, investment, or financial advice. Nothing in this report constitutes a solicitation or endorsement of a project by SafeEdges.



REVOLUTIONIZING BLOCKCHAIN AUDITS

Founded in 2023, Safe Edges is the largest blockchain security company that ensures the safety and reliability of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative technology, we support the success of our clients with best-in-class security, all whilst realizing our overarching vision: ensuring blockchain remains secure and trustworthy.

THANK YOU FOR WORKING WITH US



SAFEEDGES.IN